

Aufgabe

Sie arbeiten für CIA (Center for Investigation on Astronautics) und sind mit folgender Aufgabe beauftragt:

Es wird geplant, mehrere Satelliten zu fernen Sternen zu schicken. Es ist interessant, ob diese Satelliten, nachdem sie den Stern erreicht haben, um den Stern kreisen werden, mit ihm kollidieren oder einfach weiterfliegen. Dies ist von der Masse des Sterns **M**, von seinem Radius **R**, von der Geschwindigkeit des Satelliten **V** und von der Gravitationskonstante **G** abhängig. Die **M**, **R**, **V**, **G** sind die Fließkommazahlen.

Zuerst wird die Erste Kosmische Geschwindigkeit **EKG** berechnet:

$$\text{EKG} = \sqrt{G \cdot \frac{M}{R}}, \quad G = 6,67 \cdot 10^{-11}$$

Danach wird die Zweite Kosmische Geschwindigkeit **ZKG** berechnet:

$$\text{ZKG} = \text{EKG} \cdot \sqrt{2}$$

Die berechneten **EKG** und **ZKG** sind mit der Geschwindigkeit des Satelliten **V** zu vergleichen. Folgende Situationen sind zu programmieren:

Ist **V** < **EKG**, dann wird der Satellit mit dem Stern kollidieren.

Liegt der Wert von **V** zwischen **EKG** und **ZKG** (inkl. Grenzen), dann kreist der Satellit ewig um den Stern.

Ist **V** > **ZKG**, dann passiert der Satellit den Stern und fliegt weiter.

Sie erstellen vier Tabellen:

```
PA_STERNE      = {[ Stern:string, Masse:real, Radius:real ]}
PA_SATELLITEN  = {[ Kennung:string, Geschwindigkeit:real ]}
PA_REFERENZ    = {[ EntscheidungID:integer, Entscheidung:string ]}
PA_RESULT      = {[ Stern:string, Kennung:string, EntscheidungID:integer ]}
```

Die ersten zwei Tabellen füllen Sie selbst mit eigenen Daten (5 bis 6 Sterne, 3 bis 4 Satelliten).

Die dritte Tabelle enthält die möglichen Resultate (Entscheidungen) in kodierter Form und sollte in etwa so aussehen:

ID	Entscheidung
0	'Kreisen'
1	'Kollidieren'
2	'Weiter fliegen'
9	'Entscheidungsfehler'

Diese Tabelle ist bequem, wenn diese Anwendung durch Behörden in anderen (nicht deutschsprachigen) Ländern verwendet werden soll. Sie verwenden hier entweder deutsche oder englische Sprache. Die Kodierungen dieser Entscheidungen (Spalte ID) sind aber festgeschrieben.

Die letzte Tabelle bleibt anfangs leer — sie wird von Ihrem PL/SQL-Code befüllt.

Aufgaben:

1. Erstellen Sie ein ERM in Peter-Chen-Notation, das diesem Anwendungsfall entspricht. [7]
2. Konvertieren Sie dieses ERM in die Tabellen laut den Regeln aus dem Unterricht. [6]
3. Bringen Sie die Tabellen in 3NF und begründen die NF für jede Tabelle. [8]
4. Schreiben Sie Anweisungen (SQL-Skript, Textdatei mit CREATE TABLE... INSERT INTO...), um alle Tabellen zu erstellen und die ersten drei Tabelle mit Daten zu füllen. [6]
5. Ihre PL/SQL-Prozeduren gestalten Sie als ein Paket namens **SAT**. Die Gravitationskonstante und die Quadratwurzel aus 2 müssen als Konstanten in der Spezifikation des Pakets definiert werden. [8]
6. Schreiben Sie eine private PL/SQL-Prozedur **GetVelocity**.
Parameter: EKG (out), ZKG (out), Masse (in), Radius (in).
Funktionalität: Die EKG und ZKG werden laut den Formeln berechnet und beide auf einmal zurückgeben. [4]
7. Schreiben Sie eine public PL/SQL-Prozedur **Action**.
Parameter: Keine.
Funktionalität: Anhand von den ersten zwei Tabellen müssen für jeden Stern und für jeden Satellit durch den Aufruf der Prozedur **GetVelocity** die EKG und ZKG berechnet werden. Durch den Vergleich EKG und ZKG mit eigener Geschwindigkeit des Satelliten muss entschieden werden, ob dieser Satellit um diesen Stern ewig kreist, oder kollidiert mit ihm, oder fliegt vorbei. Diese Prozedur schreibt den ganzzahligen Kode der Entscheidung, sowie den Namen des Sternes und die Kennung des Satelliten in die vierte Tabelle. [12]
8. Schreiben Sie einen anonymen PL/SQL-Block, in dem Sie zuerst den Inhalt der Tabelle PA_RESULT löschen, dann die Prozedur **Action** aufrufen und somit automatisch die vierte Tabelle füllen. Dann öffnen Sie einen Cursor als Verknüpfung von dritter und vierter Tabelle und zeigen auf dem Bildschirm die Felder Stern, Satellit und Entscheidung. [9]

Zusätzliche Anforderungen:

Ihre Programme laut Strukturierungsprinzipien schreiben – keine exit, break, continue, return in Schleifen oder in if-Anweisungen, alle Schleifen nur durch Bedingung verlassen; nur einen return am Ende einer Funktion schreiben, kein vorzeitiges Verlassen von Prozedur/Funktion; keine goto-Anweisungen und somit keine Sprungmarken (Labels); Inhalt eines Blocks nach rechts verschieben. Genauer über die Strukturierung schauen Sie die Web-Seite azdom.de/str. Beim Nichtbeachten dieser Anforderungen werden Punkte abgezogen (ein Punkt für jede Verletzung der Strukturierung).

Testdaten:

Tabelle PA_SATELLITEN:

KENNUNG	GESCHWINDIGKEIT
Bohr	9.90E+04
Galileo	5.00E+05
Higgs	1.28E+14
Kopernikus	1.31E+08
Newton	9.10E+03
Plank	7.77E+78

Tabelle PA_STERNE:

STERN	MASSE	RADIUS
Aldebaran	3.38E+30	3.07E+10
Arktur	2.19E+30	1.77E+10
Betelgeuse	3.28E+31	6.17E+11
Orion	6.20E+35	1.67E+13
Polarstern	8.70E+30	7.78E+08
Sonne	1.99E+30	6.96E+08
Erde	5.97E+24	6.37E+06